

Power Routine — como o backend funciona

Escrito para quem vai montar o diagrama do sistema. Não precisa saber Python para entender.

O que esse sistema faz, em uma frase

O usuário informa dados do corpo (sexo, data de nascimento, altura, peso) e o que quer alcançar (emagrecer, manter ou ganhar massa). A partir disso o sistema calcula **quantas calorias ele deve comer por dia** e como dividir isso entre proteína, carboidrato e gordura. Depois, todo dia o usuário registra o que realmente comeu, e o sistema mostra o quanto ele ficou perto ou longe da meta.

É isso. Todo o resto do documento é sobre *como* essa conta é feita, onde os dados ficam guardados e por que o código está dividido do jeito que está.

A conta, que é o coração do sistema

Nada é inventado: são fórmulas de nutrição consagradas, aplicadas em cadeia. Cada etapa alimenta a seguinte. **Este encadeamento é provavelmente o diagrama mais importante do trabalho.**



A cadeia de cálculo. Cada caixa é uma função separada e testável.

O que cada etapa faz

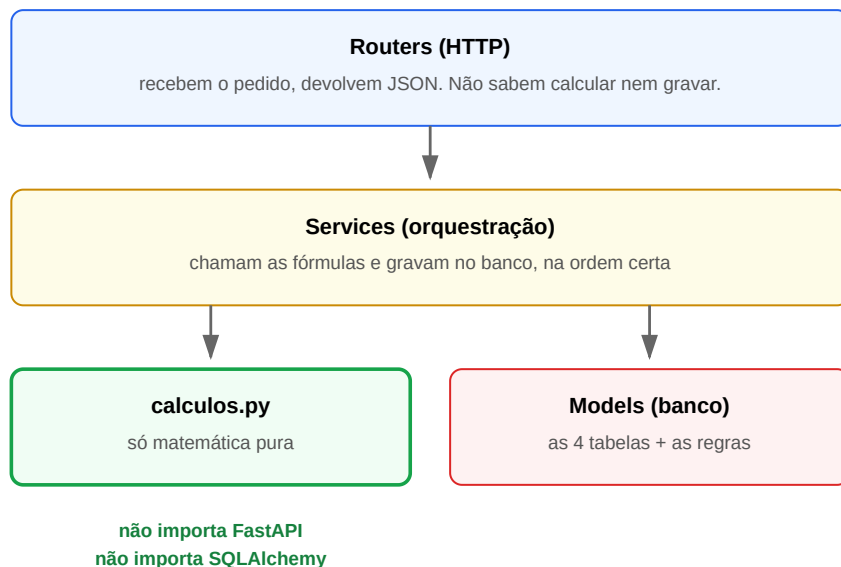
Etapa	Como funciona
Idade	Calculada da data de nascimento, não digitada. Assim ela nunca fica desatualizada.
TMB	Taxa Metabólica Basal — quantas calorias o corpo gasta em repouso absoluto. Fórmula de Harris-Benedict revisada (Roza & Shizgal, 1984), com coeficientes diferentes para homem e mulher.
GET	Gasto Energético Total — a TMB multiplicada por um fator de atividade física: sedentário 1,2 · leve 1,375 · moderado 1,55 · intenso 1,725 · muito intenso 1,9.
Meta calórica	O GET ajustado pelo objetivo: emagrecer aplica déficit de 20% ($\times 0,80$), manter não muda nada ($\times 1,00$), ganhar massa aplica superávit de 15% ($\times 1,15$).

Macros	A divisão da meta em gramas. A ordem importa: primeiro fixa a proteína (1,8 g por kg de peso; 2,0 se o objetivo for ganhar massa), depois a gordura (25% das calorias), e o carboidrato fica com o que sobrar .
---------------	--

Por que a proteína e a gordura vêm primeiro: a proteína protege a massa magra durante o déficit e a gordura sustenta a função hormonal — as duas têm um mínimo que não dá para espremer. O carboidrato é o que existe para dar energia, então é ele que absorve a sobra. Se a meta calórica for tão baixa que nem cabem proteína e gordura, o sistema **recusa** em vez de devolver um número negativo.

Como o código está dividido — e por que isso importa

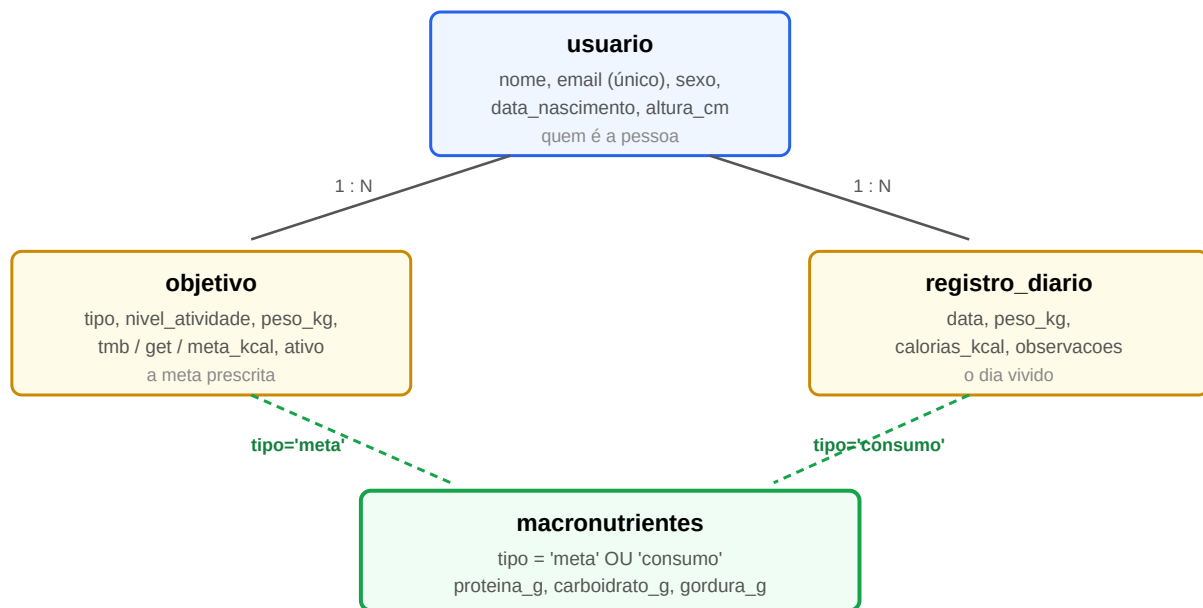
O backend tem três camadas, e a regra é que cada uma só conversa com a de baixo. Isso não é preciosismo: é o que permite testar as fórmulas de nutrição **sem banco de dados e sem servidor rodando**.



As três camadas. A seta só desce — nunca sobe.

A prova de que a separação é real: se você desligar o banco de dados e rodar os testes, **25 passam e 60 falham**. Os 25 que passam são exatamente os testes das fórmulas — eles não sabem que existe banco. Isso foi medido, não é teoria.

O banco de dados — quatro tabelas



O modelo relacional. As linhas tracejadas são o truque explicado abaixo.

As três decisões de modelagem que valem explicar

- 1. Guardamos a data de nascimento, não a idade.** Idade é um número que apodrece — se você gravar "25", em um ano estará errado e ninguém vai perceber. A data de nascimento nunca muda, e a idade é calculada na hora em que precisa.
- 2. O objetivo guarda uma fotografia do cálculo.** As colunas `tmb_kcal`, `get_kcal` e `meta_kcal` ficam gravadas no objetivo, mesmo sendo calculáveis. Isso é de propósito: é o registro do que foi prescrito naquele momento. Se amanhã a fórmula do sistema for ajustada, o histórico do usuário não muda retroativamente. E quando ele recalcula o perfil, o objetivo antigo não é apagado — só é marcado como `ativo = false`. O histórico inteiro fica.
- 3. Meta e consumo moram na mesma tabela.** Esta é a decisão mais interessante, e a que dá mais o que falar na apresentação. Existe uma única tabela `macronutrientes` com uma coluna `tipo` que diz se aquela linha é a **meta prescrita** (ligada a um objetivo) ou o **consumo real** (ligado a um dia). Poderiam ser duas tabelas separadas — mas aí comparar meta com consumo exigiria duas consultas e uma junção feita na mão. Com uma tabela só, **o comparativo sai de uma consulta única**.

E isso foi medido: a tela de comparativo dispara **4 consultas ao banco tanto com 1 dia quanto com 20 dias** registrados. O número não cresce com a quantidade de dados.

O detalhe que impede a bagunça: uma tabela que serve para duas coisas poderia acabar com linhas sem sentido — uma "meta" apontando para um dia, ou uma linha órfã. Por isso o banco tem uma regra (`CHECK`) que aceita **exatamente duas formas**: ou é `meta` e aponta só para um objetivo, ou é `consumo` e aponta só para um dia. Qualquer outra combinação o banco recusa. Não é o programa que garante isso — é o banco.

Os endpoints — o que dá para pedir à API

Rota	O que faz
POST /api/usuarios	Cadastra a pessoa. Email precisa ser único.
GET /api/usuarios/{id}	Devolve os dados e a idade já calculada.
POST /api/perfil/calcular	O principal. Recebe peso, nível de atividade e objetivo; roda a cadeia inteira e grava o resultado como o objetivo ativo da pessoa.
POST /api/diario/registro	Registra o dia: peso, calorias e macros consumidos.
GET /api/diario/{id}	O comparativo. Lista os dias do mais recente ao mais antigo, cada um ao lado da meta vigente, com a diferença em calorias e o percentual de aderência.
GET /api/saude	Só diz se a API está no ar.

Um detalhe do diário que vale saber

Registrar o mesmo dia duas vezes **não cria dois registros** — o segundo envio regravava o primeiro. Isso é proposital: o alternativo seria o app ter que descobrir se o dia já existe antes de enviar. Mas atenção: a regravação **substitui o dia inteiro**, não faz mesclagem. Se o segundo envio omitir a observação, a observação anterior é apagada. O app precisa sempre mandar o dia completo.

Como um pedido é tratado, do começo ao fim

Vale um diagrama de sequência. O caminho de `POST /api/perfil/calcular`:

1. O **router** recebe o JSON. Antes de qualquer coisa, o **Pydantic** confere formato e faixas: peso entre 20 e 400 kg, nível de atividade tem que ser um dos cinco válidos, e assim por diante. Se falhar, para aqui com **422**.
2. Abre-se **uma transação** no banco. Uma só, para o pedido inteiro.
3. O **service** busca o usuário. Não existe? **404**.
4. Chama a cadeia de cálculo. Se a meta não couber nos macros, **422** — e nada foi gravado ainda, de propósito: *calcula antes de escrever*, para um erro nunca deixar a pessoa sem objetivo.
5. Desativa o objetivo anterior, grava o novo e a linha de meta.
6. Se o banco recusar alguma regra, vira **409**.
7. Deu tudo certo: a transação confirma *uma vez só*, e o router devolve o JSON.

As três camadas de validação

Camada	O que ela barra	Resposta
Pydantic (entrada)	formato errado, número fora da faixa, valor inválido, data no futuro	422
Service (regra de negócio)	usuário que não existe, meta calórica impossível	404 / 422
PostgreSQL (integridade)	email duplicado, dois objetivos ativos, linha de macro malformada	409

O ponto: **nenhum router tem tratamento de erro**. Todos os erros são traduzidos em código HTTP num único arquivo. Se amanhã quiser mudar o que um erro devolve, muda em um lugar só.

Uma sutileza que custou caro descobrir: o service precisa mandar a escrita para o banco *durante* o pedido, e não deixar para o final. Se a violação de regra só aparecer no momento de confirmar a transação, a resposta HTTP já começou a ser enviada e o erro vira um **500 genérico** em vez do 409 correto. Foi medido em quatro pontos diferentes do sistema.

Exemplo real, com os números que o sistema devolve

Homem, 25 anos, 80 kg, 1,80 m, atividade moderada, quer emagrecer:

```
POST /api/perfil/calcular
{ "usuario_id": 2, "peso_kg": 80, "nivel_atividade": "moderado",
  "objetivo": "emagrecer", "peso_meta_kg": 72 }

→ { "tmb_kcal": 1882.02,      ← gasto em repouso
    "get_kcal": 2917.13,     ← × 1,55 (moderado)
    "meta_kcal": 2333.70,    ← × 0,80 (déficit de 20%)
    "macros": { "proteina_g": 144.00,      ← 1,8 g × 80 kg
                  "carboidrato_g": 293.57, ← o que sobrou
                  "gordura_g": 64.82 } }    ← 25% das calorias
```

Depois de registrar dois dias, o comparativo:

```
GET /api/diario/2

→ { "objetivo": "emagrecer", "meta_kcal": 2333.7,
    "registros": [
      { "data": "2026-09-04", "consumido_kcal": 2450.0,
        "diferenca_kcal": 116.3, "aderencia_percentual": 104.98 },
      { "data": "2026-09-03", "consumido_kcal": 2300.0,
        "diferenca_kcal": -33.7, "aderencia_percentual": 98.56 }
    ] }
```

Cada dia vem com os macros consumidos ao lado dos macros da meta, para a comparação ser direta. A ordem é do mais recente para o mais antigo.

Tecnologias, para a caixinha do diagrama

Linguagem	Python 3.12
API	FastAPI — gera sozinho a documentação interativa (Swagger)
Validação de entrada	Pydantic v2
Acesso ao banco	SQLAlchemy 2
Versionamento do banco	Alembic — cada mudança de estrutura é um arquivo versionado
Banco	PostgreSQL 16, rodando em container Docker
Testes	pytest — 85 testes , todos passando

Se for desenhar só um diagrama, desenhe a cadeia de cálculo (idade → TMB → GET → meta → macros). É ela que explica o que o sistema faz. Se der para desenhar dois, o segundo é o modelo de dados com a tabela **macronutrientes** servindo aos dois lados — é a decisão de projeto que mais rende explicação.